choosing filters

Table of contents

packages required 1
data prep 1
function to calculate all metric used to compare filters 2
analyse of the filtered datasets 3
Advanced visualisation for datasets filtered with IQR 5

packages required

This analysis depends on the following packages:

data prep

The dataset is loaded and curated. We remove repetitions where adult = 0 as well as all rows containing NAs. This is justified because we believe that adult mortality is linked to issues occurring during installation rather than a response from the plant. Since adult death directly affects fecundity measures, keeping these cases would introduce substantial bias.

```
data250_afabae_00 <- read.csv("../../../../../../01.data/2023_screen_specifics_251genotypes/0
  mutate(ivip = as.factor(ivip)) %>%
  mutate(session = as.factor(session))

data250_afabae_01 <- data250_afabae_00 %>% filter(!is.na(larvae_total_number))
data250_afabae_01$ivip <- droplevels(data250_afabae_01$ivip)

data250_afabae_02<- data250_afabae_01 %>% filter(adult_alive != "0") #remove if no adult
data250_afabae_02$ivip <- droplevels(data250_afabae_02$ivip)

data250_afabae_02$ivip <-as.factor(data250_afabae_02$ivip)
```

Once we have removed the NAs and all rows for which we don't have an adult, we can still make adjustments to the dataset. We can remove outliers to ensure that our distribution approaches a normal distribution, and we can also filter out genotypes based on a threshold for the minimum number of repetitions required to keep a genotype. We have 3 methods to handle outliers: 1)keep the outliers; 2)remove manually the outliers based on biological reasons; 3) use the interquartile range method (IQR) to remove all data outside a specific range.

The goal of this script is to study how our model reacts to changes in the outlier removal method and the minimum number of repetitions required to retain a genotype.

in the folowing loop we varry two parameters, i the minimal number of reps for the genotypes and the outlier filtering method.

function to calculate all metric used to compare filters

```
analyze_filtered_data <- function(data, i, filter_name) {
  data <- data %>%
    group_by(ivip) %>%
    filter(n() >= i) %>%
    ungroup() %>%
    droplevels()

  if (nrow(data) < 5 || nlevels(data$ivip) < 2) {
    return(NULL)  # skip iterations with too little data
  }

  NGEN <- nlevels(data$ivip)
  n_after <- nrow(data)

  # Fit the model
  mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)
  residuals_values <- residuals(mod)
  RMSE <- sqrt(mean(residuals_values^2))
  shapiro_res <- shapiro.test(residuals_values)$p.value
  shapiro <- shapiro.test(data$larvae_total_number)$p.value

  # Heritability model
  mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)
  estimates <- as.data.frame(VarCorr(mod_h2))
  genetic_variance <- estimates$vcov[1]
  residual_variance <- estimates$vcov[3]
  nbE <- nlevels(data$session)
```

```
  H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))

  return(data.frame(
    filter = filter_name, i = i, RMSE = RMSE, n_after = n_after,
    H2 = H2, shapiro = shapiro, shapiro_res = shapiro_res, NGEN = NGEN
  ))
}
```

analyse of the filtered datasets

```
# Initialize an empty data frame to store results
results <- data.frame(x = numeric(0), i = numeric(0), RMSE = numeric(0),
                      n_after = numeric(0), H2=numeric(0),shapiro=numeric(0),shapiro_res=nu


###filter based on IQR
    # Calculate IQR and bounds for outlier removal
    IQR_value <- IQR(data250_afabae_02$larvae_total_number)
    lower_bound <- quantile(data250_afabae_02$larvae_total_number, 0.25) - 1.5 * IQR_value
    upper_bound <- quantile(data250_afabae_02$larvae_total_number, 0.75) + 1.5 * IQR_value
    filter ="IQR"
    # Filter out outliers
    data_iqr <- data250_afabae_02[data250_afabae_02$larvae_total_number >= lower_bound & data


for (i in 2:9) {
  res <- analyze_filtered_data(data_iqr, i, filter_name = "IQR")
  if (!is.null(res)) results <- rbind(results, res)
}

for (i in 2:9) {
  res <- analyze_filtered_data(data250_afabae_02, i, filter_name = "no_filter")
  if (!is.null(res)) results <- rbind(results, res)
    }


  # Identify the indices of the top 3 highest values
top_3_indices <- order(data250_afabae_02$larvae_total_number, decreasing = TRUE)[1:3]
    # View the top 3 highest values
data250_afabae_02[top_3_indices, ] %>% dplyr::select(genotype, ivip, aphid, larvae_total_numb
```

```
     genotype       ivip          aphid larvae_total_number
1146      164 IVIP101838 A.fabae AF001                 100
1089      155 IVIP101738 A.fabae AF001                  84
123        19 IVIP100234 A.fabae AF001                  77
```

```
# Remove the rows with the top 3 highest values
data250_afabae_03 <- data250_afabae_02[-top_3_indices, ]

data_manual<- data250_afabae_03 %>% filter(larvae_total_number != "0")

for (i in 2:9) {
  res <- analyze_filtered_data(data_manual, i, filter_name = "Manual_filter")
  if (!is.null(res)) results <- rbind(results, res)
}

print(results)
```

```
          filter i      RMSE n_after        H2      shapiro  shapiro_res NGEN
1            IQR 2  9.233038    1735 0.3447490 5.266510e-11 1.483460e-08  215
2            IQR 3  9.229700    1729 0.3446498 5.543759e-11 1.270126e-08  212
3            IQR 4  9.226764    1714 0.3428978 7.173332e-11 1.658017e-08  207
4            IQR 5  9.261827    1686 0.3366220 1.084763e-10 1.885992e-08  200
5            IQR 6  9.344007    1621 0.3188691 2.352926e-10 1.694455e-08  187
6            IQR 7  9.363300    1519 0.2900334 6.309009e-10 1.945748e-08  170
7            IQR 8  9.474470    1309 0.2334308 8.692906e-09 1.509873e-08  140
8            IQR 9  9.729894     917 0.2094154 8.087895e-07 2.549500e-06   91
9      no_filter 2 10.256889    1766 0.2957885 3.470226e-14 1.326403e-13  215
10     no_filter 3 10.258422    1762 0.2964816 3.127027e-14 1.274759e-13  213
11     no_filter 4 10.216287    1747 0.3001616 2.337939e-14 1.340294e-13  208
12     no_filter 5 10.220998    1719 0.2985800 5.058276e-14 2.058611e-13  201
13     no_filter 6 10.284203    1654 0.2870996 1.875066e-13 7.813558e-13  188
14     no_filter 7 10.339821    1570 0.2682935 5.961660e-13 1.494831e-12  174
15     no_filter 8 10.373956    1367 0.2267790 3.134460e-12 3.314494e-12  145
16     no_filter 9 10.644708     999 0.2130584 6.541015e-10 3.209480e-10   99
17 Manual_filter 2  9.634853    1750 0.3377797 4.862441e-11 3.124783e-10  215
18 Manual_filter 3  9.635051    1746 0.3384563 4.206604e-11 2.953583e-10  213
19 Manual_filter 4  9.584852    1728 0.3444121 2.348789e-11 1.660786e-10  207
20 Manual_filter 5  9.625195    1700 0.3385120 3.469540e-11 1.947332e-10  200
21 Manual_filter 6  9.724575    1640 0.3199037 8.005854e-11 2.647481e-10  188
22 Manual_filter 7  9.760221    1544 0.3039618 4.653498e-10 5.248763e-10  172
23 Manual_filter 8  9.839597    1334 0.2571395 4.052453e-09 7.448826e-10  142
24 Manual_filter 9 10.119513     966 0.2086034 5.997644e-07 9.699518e-08   96
```

Based on these results it seems appropriate to select the IQR filter as it yield best heritability, lower RMSE and more normal data.

Advanced visualisation for datasets filtered with IQR

```
write.csv(x = results,
          file = "filter_annalysis.csv",
          quote = F, row.names = F)
```

```
iqr_results <- results %>% filter(filter == "IQR")
```

We now need to visualize the data we generated from the IQR filter to choose the best value for i (minimal number of repetitions to keep a genotype in the analysis). The code will remove all genotypes for which we have less than i repetitions. The choice or i is complex as a higher value for i means that we have less genotypes in the GWAS but the genotypes we have have more repetition. However with a lower value for i we have more genotypes in the annalysis but some will have low number of repetition. Our objective is to find the optimal value to maximise the GWAS power without adding too much noise. We metrics we use to choose based on overall number of genotypes, total number of data points, RMSE and heritability.

Based on these first results it appears that IQR is the best filtering option. We still need to determine the best value for i.

```
# Initialize an empty data frame to store results
results <- data.frame(x = numeric(0), i = numeric(0), RMSE = numeric(0),
                      n_after = numeric(0), H2=numeric(0),shapiro=numeric(0),shapiro_res=nu


for (i in 2:9) {
  ###filter based on IQR
    # Calculate IQR and bounds for outlier removal
    IQR_value <- IQR(data250_afabae_02$larvae_total_number)
    lower_bound <- quantile(data250_afabae_02$larvae_total_number, 0.25) - 1.5 * IQR_value
    upper_bound <- quantile(data250_afabae_02$larvae_total_number, 0.75) + 1.5 * IQR_value
    filter ="IQR"
    # Filter out outliers
    data <- data250_afabae_02[data250_afabae_02$larvae_total_number >= lower_bound & data250


    data <- data %>%
      group_by(ivip) %>%           # Group by genotype
```

```
  filter(n() >= i) %>%             # Keep only groups with i or more repetitions
  ungroup()


data$ivip <- droplevels(data$ivip)

NGEN<-nlevels(data$ivip)




# Number of rows after removing outliers
n_after <- nrow(data)



# Fit the model
mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)

# Extract fitted values and residuals
fitted_values <- fitted(mod)
residuals_values <- residuals(mod)

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(residuals_values)

# Print the p-value (this is your quantification of normality)
shapiro_res <- shapiro_result$p.value

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(data$larvae_total_number)

# Print the p-value (this is your quantification of normality)
shapiro <- shapiro_result$p.value


# Calculate RMSE (Root Mean Squared Error)
RMSE <- sqrt(mean(residuals_values^2))

# Fit the model with a fixed formula
mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)
```

```
    # Get the variance components
    estimates <- as.data.frame(VarCorr(mod_h2))

    # Extract the relevant variance components
    genetic_variance <- estimates$vcov[1]
    residual_variance <- estimates$vcov[3]  # Adjust based on the placement of residual varia

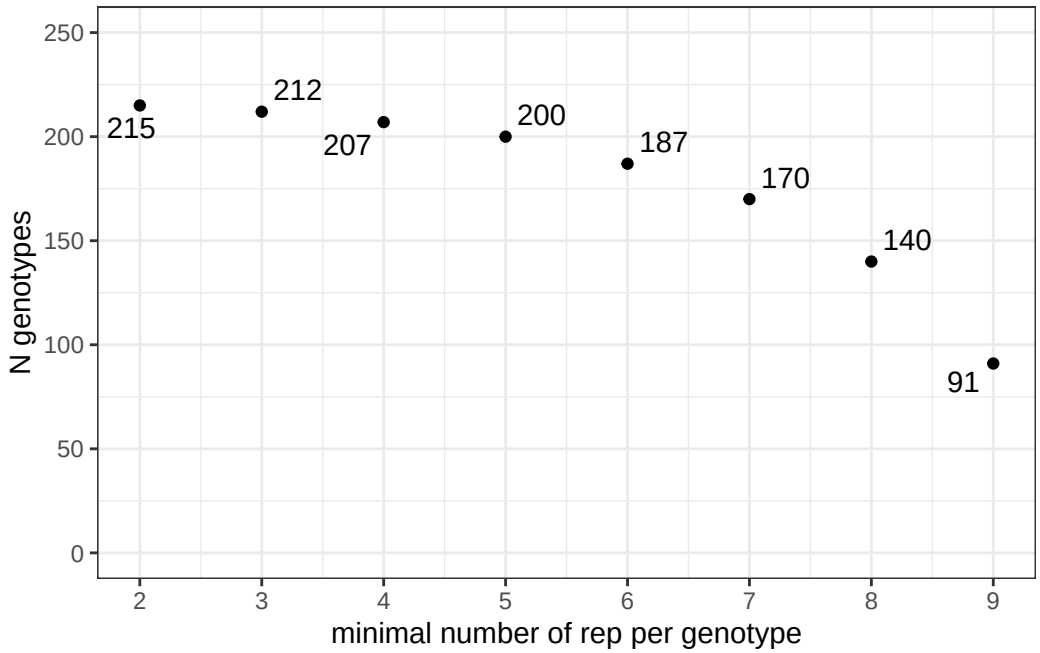
    nbE <-nlevels(data$session)

    # Calculate heritability (h2)
    H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))


    # Store the results in the data frame
    results <- rbind(results, data.frame(filter = filter, i = i, RMSE = RMSE, n_after = n_af
}
```

We now need to visualize the data we generated to choose the best filter

```
ggplot(results, aes(x = i, y = NGEN)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,250)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "minimal number of rep per genotype", y = "N genotypes")
```

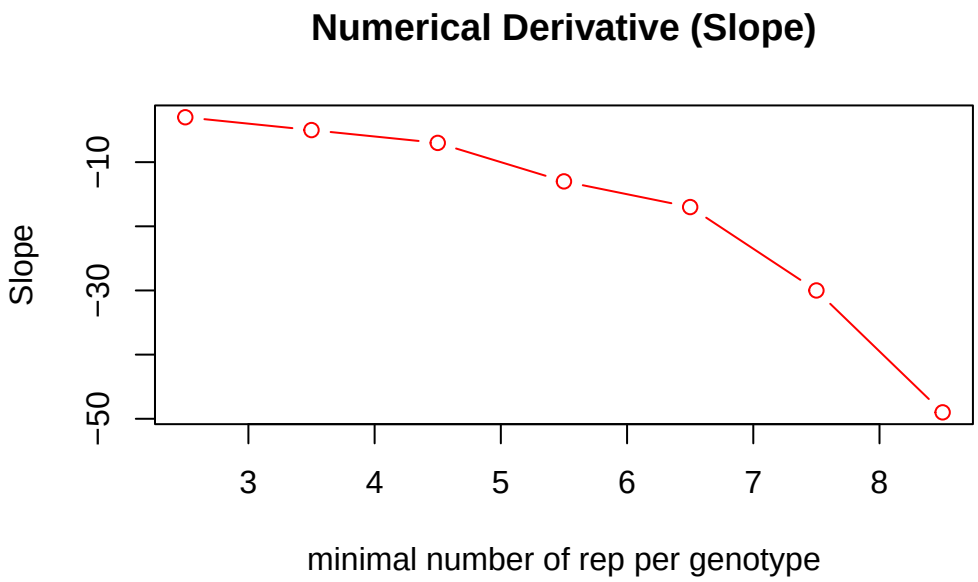



```r
y<-as.vector(results$NGEN)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

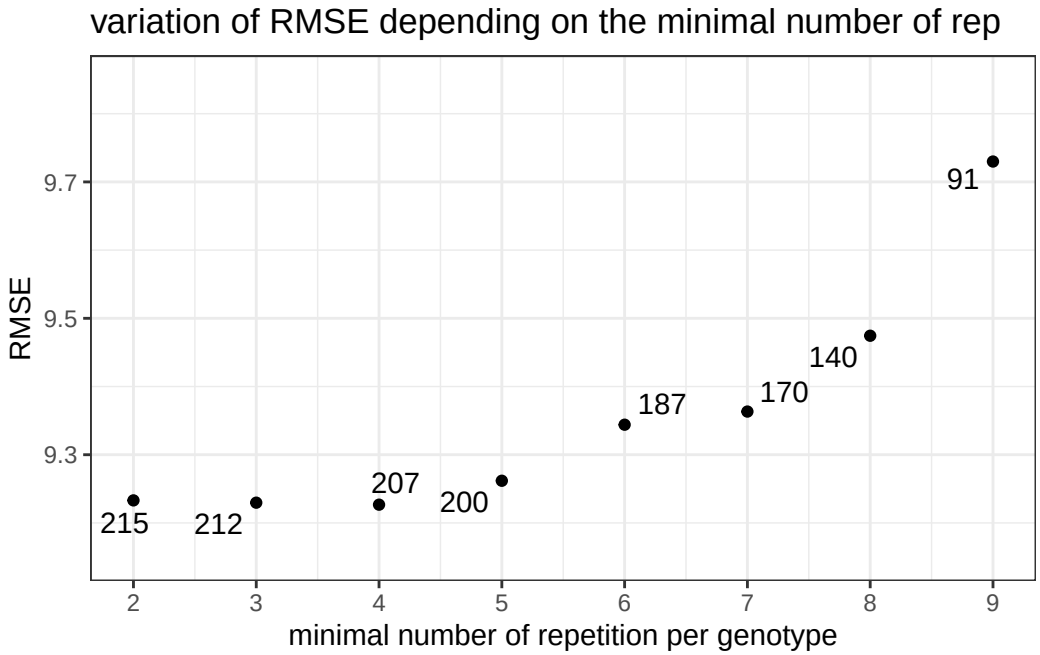
# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "minimal number of rep per genotype")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



It can be hard to read the relation between Genotype number and i. Therefore it is helpfull to plot the slop (the value is the difference between number of genotype with i and i + 1). From 2 to 5 the slope is progressive then we start genotypes with a higher rate.

```
ggplot(results, aes(x = i, y = RMSE)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(9.15,9.85)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "minimal number of repetition per genotype", y = "RMSE") +
  ggtitle("variation of RMSE depending on the minimal number of rep")
```

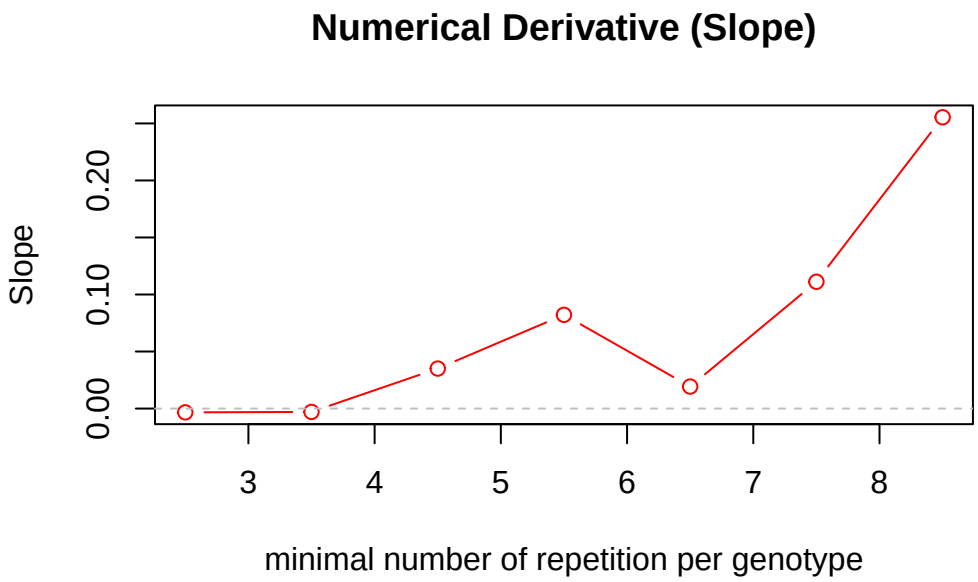



```r
y<-as.vector(results$RMSE)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

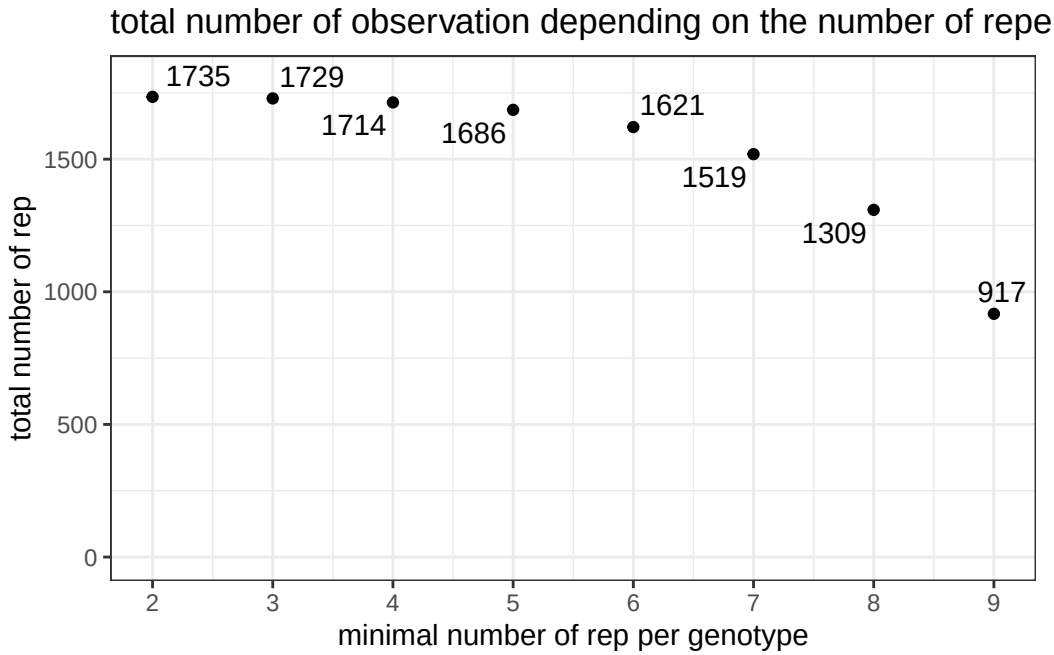
# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "minimal number of repetition per genotype")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



the lowest value for RMSE is with i=4.

```
ggplot(results, aes(x = i, y = n_after)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,1800)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = n_after), arrow = T) +
  labs(x = "minimal number of rep per genotype", y = "total number of rep") +
  ggtitle("total number of observation depending on the number of repetitions")
```

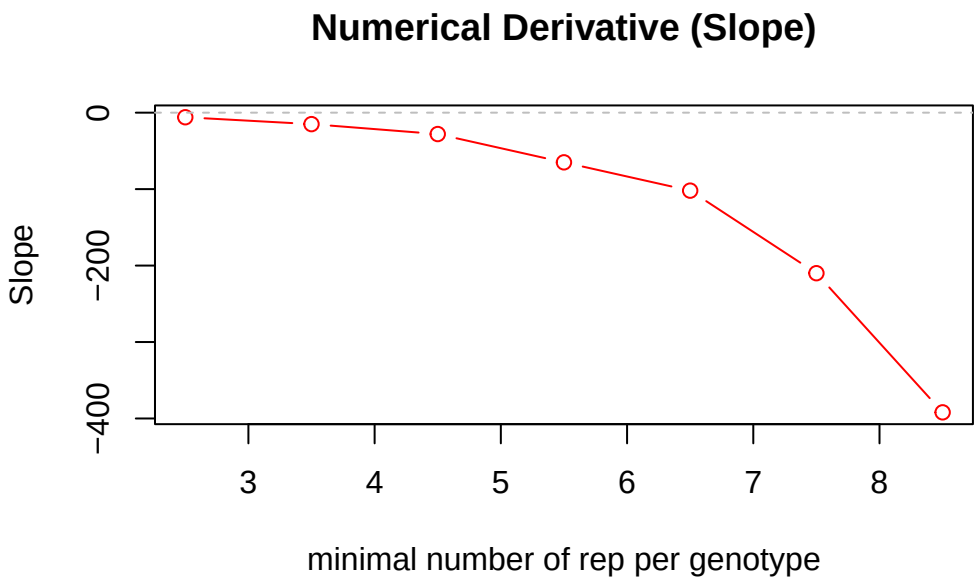



```r
y<-as.vector(results$n_after)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

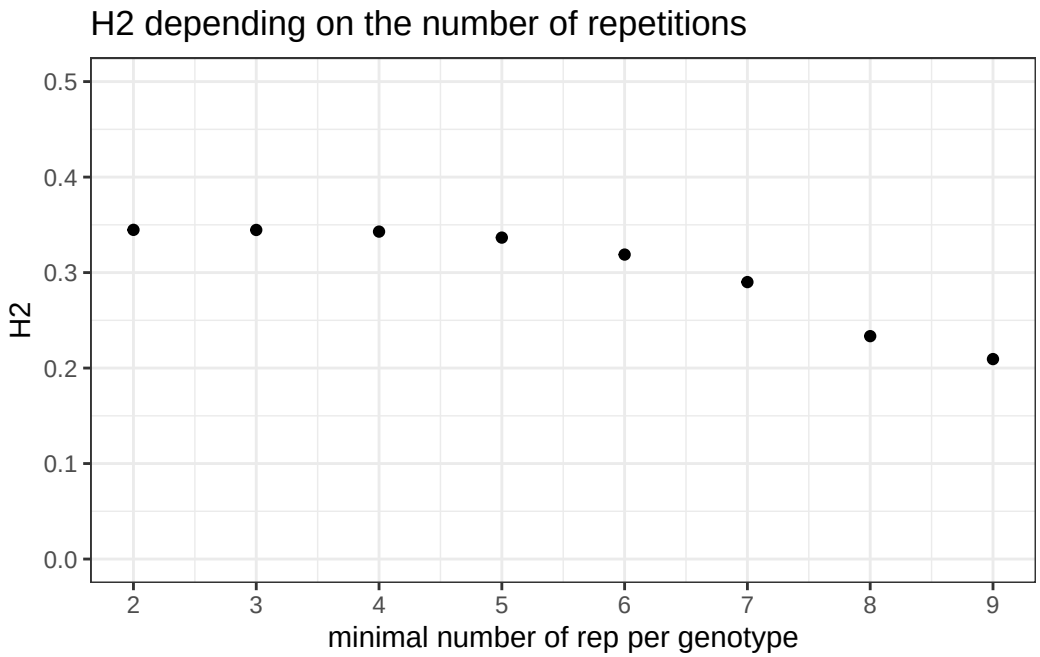
# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "minimal number of rep per genotype")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



The total number or repetition is the total number of line in our dataset (the number of genotypes x number of repetition per genotype). This metric slowly decrease as i increase. For a i value of 2 to 4 decrease we lose only few repetition, then from 6 to 9 the slope increase and the rate of repetition loss increase.

```
ggplot(results, aes(x = i, y = H2)) +
  geom_point() +
  scale_x_continuous(breaks = seq(1, 13, 1)) +
  scale_y_continuous(limits = c(0, 0.5)) +
  theme_bw() +
  labs(
    x = "minimal number of rep per genotype",
    y = "H2"
  ) +
  ggtitle("H2 depending on the number of repetitions")
```

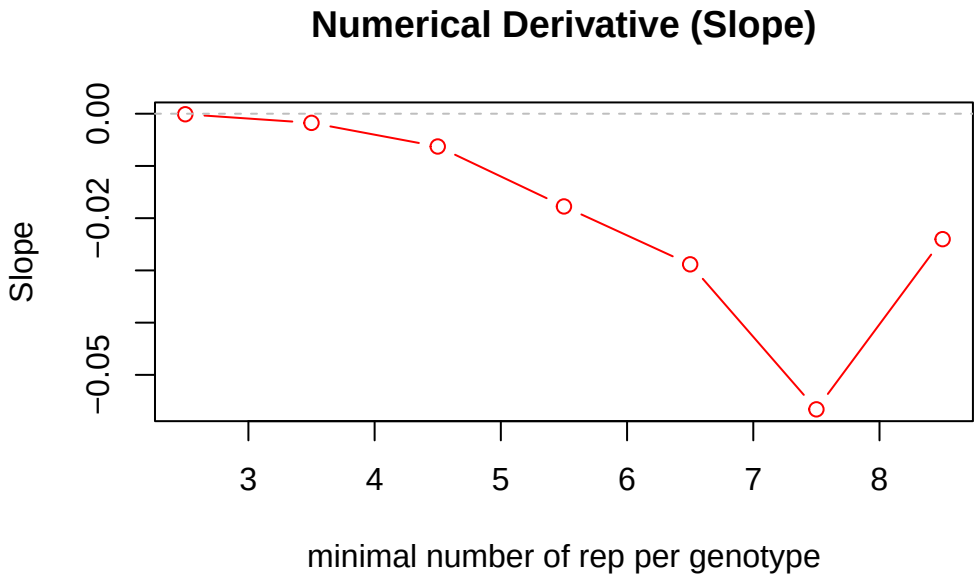



```r
y<-as.vector(results$H2)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "minimal number of rep per genotype")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



the highest heritabilty are obtained with i=2, 3 or 4. Heritability then progresivelly decrease as we increase the minimal number of repetitions required.

Based on all the metrics presented here it seems that a value of 4 or less would be optimal. choosing 5 could be tempting as we don't lose too much heritability and we don't gain too much RMSE but due to the already low number of genotypes reducing the genotype loss is our priority. However we think that 4 is the minimal number of acceptable repetitions. Going lower to 4 would be quite risky especially since we know that our phenotypes is quite variable. Therefore we choose to keep 4 as the ideal value for i. In the next steps we will conduct GWAS on dataset filterd with a i value of 4, 5 and 6. This is intended to make sure that the choice of i does not change the GWAS pics the detect, however we will only use the GWAS pics from the dataset with i=4.

We know have to make sure this filtering does not include some bias. We want to asses this throught 2 key points, the resistance level and the genetic cluster.

```
# ------------------------------
# 1. IQR filtering
# ------------------------------
IQR_value <- IQR(data250_afabae_02$larvae_total_number)
lower_bound <- quantile(data250_afabae_02$larvae_total_number, 0.25) - 1.5 * IQR_value
upper_bound <- quantile(data250_afabae_02$larvae_total_number, 0.75) + 1.5 * IQR_value

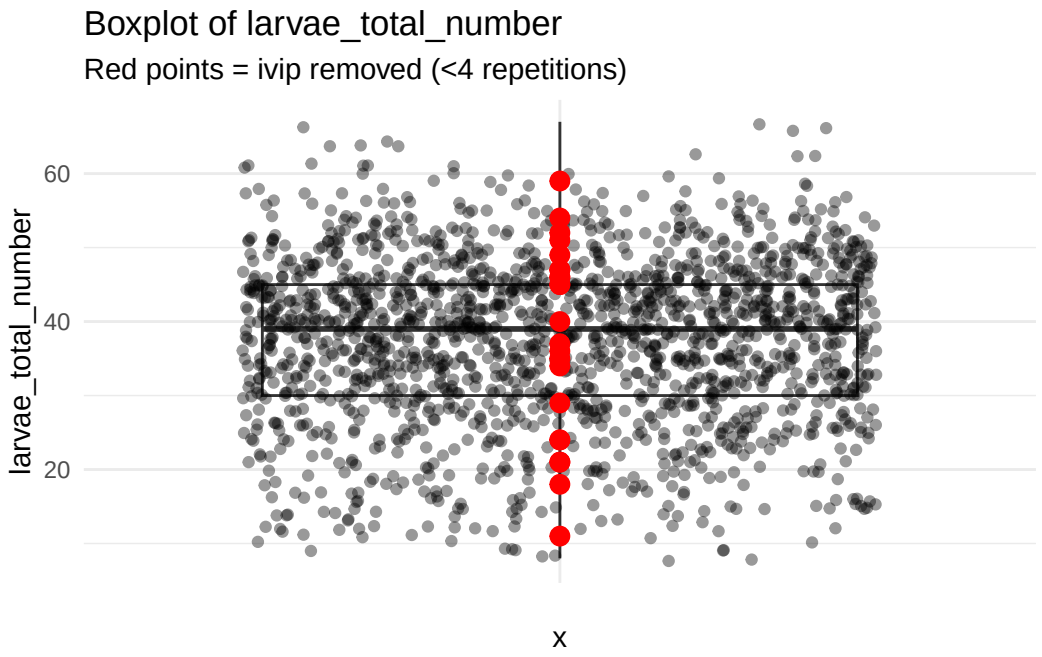
data <- data250_afabae_02 %>%
```

```r
  filter(larvae_total_number >= lower_bound,
         larvae_total_number <= upper_bound)

# ------------------------------
# 2. Keep ivip with  4 repetitions
# ------------------------------
data_afabae_4 <- data %>%
  group_by(ivip) %>%
  filter(n() >= 4) %>%
  ungroup()

# ------------------------------
# 3. Identify ivip NOT kept
# ------------------------------
ivip_not_in_4 <- setdiff(unique(data$ivip), unique(data_afabae_4$ivip))
data_removed <- data %>% filter(ivip %in% ivip_not_in_4)

# ------------------------------
# 4. Boxplot with removed ivip in red
# ------------------------------
ggplot(data, aes(x = "", y = larvae_total_number)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(alpha = 0.4) +
  geom_point(data = data_removed,
             aes(x = "", y = larvae_total_number),
             color = "red",
             size = 3) +
  labs(title = "Boxplot of larvae_total_number",
       subtitle = "Red points = ivip removed (<4 repetitions)") +
  theme_minimal()
```




```
# ------------------------------
# 5. Table of removed ivip with genetic cluster
# ------------------------------
info_cluster <- read.csv("../../../../../../00.info/11.infos_genotypes/morphological_phenotl

ivip_removed_cluster_table <- data.frame(ivip = ivip_not_in_4) %>%
  left_join(info_cluster, by = "ivip")

ivip_removed_cluster_table
```

```
         ivip Accession.code                       Nom code.synonyme
1  IVIP100127        DVF0127                     FRIBO   IPK FAB 204
2  IVIP100225        DVF0225               BURCHTINSKI
3  IVIP100279        DVF0279             LORRAINE METZ
4  IVIP100328        DVF0328                       38D
5  IVIP101289        DVF1289                 MGD0110656
6  IVIP101454        DVF1454 KORNBERGER KLEINKOERNIGE    IPK FAB242
7  IVIP101536        DVF1536                     894 B
8  IVIP101777        DVF1777                    ASCOTT
9  IVIP102191        DVF2191                       STW
10 IVIP102220        DVF2220                    K 0285
   DATE.RECEPTION     Statut.biologique.SIREGAL Origine.Géographique
```

```
1        19.11.69 Advanced or improved cultivar              GERMANY
2        06.04.70          Traditional landrace             BULGARIA
3        10.04.71          Traditional landrace               FRANCE
4        18.05.72          Traditional landrace             ETHIOPIA
5        03.04.82          Traditional landrace                YEMEN
6        31.01.84 Advanced or improved cultivar              AUSTRIA
7        07.12.87    Breeding/research material                 PERU
8        01.01.04 Advanced or improved cultivar               FRANCE
9        01.03.01            Spontaneous mutant                SUDAN
10       01.06.01          Traditional landrace                CHINA
   HIVER.PRTPS    INTERET..Caractères.spéciaux  ESPECE.et.sous.espèce TANNINS
1            P                                 Vicia faba var. equina       T
2            P                                 Vicia faba var. equina       T
3            P                    Sens. froid  Vicia faba var. minor       T
4            P                                  Vicia faba var. minor       T
5                          folioles étroits  Vicia faba var. minor       T
6            P                                  Vicia faba var. minor       T
7            P téguments avec tâches instables  Vicia faba var. major       T
8            P                                  Vicia faba var. minor       T
9            P R. fusarium, early, autofertile Vicia faba var. equina      ZT
10           P                                 Vicia faba var. equina       T
   SumTem                                STATUT.Biologique  cluster_name
1  958.15 trad_CULTIVAR / Advanced or improved cultivar ?         North
2  902.35                                     trad_LANDRACE         North
3  995.90                                     trad_LANDRACE         North
4  902.35                                     trad_LANDRACE         South
5  884.95                                     trad_LANDRACE         South
6  978.30                              ADV./IMPR. CULTIVAR         North
7  978.30                          BREED./RESEARCH MATERIAL South_America
8  827.55                              ADV./IMPR. CULTIVAR         North
9  782.45                                           mutant         South
10 920.05                                     trad_LANDRACE          East
```

```r
# ------------------------------
# 6. Cluster summary table
# ------------------------------
# Keep only non-empty, non-NA cluster_name
ivip_with_cluster <- data %>%
  distinct(ivip) %>%
  left_join(info_cluster, by = "ivip") %>%
  filter(!is.na(cluster_name), cluster_name != "")
```

```r
cluster_summary <- ivip_with_cluster %>%
  group_by(cluster_name) %>%
  summarise(
    n_original = n(),
    n_removed = sum(ivip %in% ivip_not_in_4)
  ) %>%
  arrange(cluster_name)

cluster_summary
```

```
# A tibble: 6 x 3
  cluster_name  n_original n_removed
  <chr>              <int>     <int>
1 East                  23         1
2 North                 88         5
3 Paucijuga              5         0
4 South                 68         3
5 South_America         16         1
6 Winter                16         0
```

based on these results it does not seems that we have a cluster or resistance level bias